

Financial Fraud Detection

Introduction

Financial Fraud Detection is the process of monitoring transactions and customer behavior to identify and stop fraudulent activity. According to Juniper Research's 2022 study about online payment fraud, globally payment fraud are noted to exceed \$343 billion between 2023 and 2027.

Traditionally, firms have used fraud detection and prevention to curb company financial losses and maintain positive customer relations.

```
In [6]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, recall_score, precision_score, classifi
```

Reading the data

```
In [8]: df = pd.read_csv("Synthetic_Financial_datasets_log.csv")
```

Printing the data

```
In [9]: df.head()
```

```
Out[9]:
```

	step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	nameDest
0	1	PAYMENT	9839.64	C1231006815	170136.0	160296.36	M1979787155
1	1	PAYMENT	1864.28	C1666544295	21249.0	19384.72	M2044282225
2	1	TRANSFER	181.00	C1305486145	181.0	0.00	C553264065
3	1	CASH_OUT	181.00	C840083671	181.0	0.00	C38997010
4	1	PAYMENT	11668.14	C2048537720	41554.0	29885.86	M1230701703

Statistical analysis

```
In [10]: df.shape
```

```
Out[10]: (6362620, 11)
```

```
In [12]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6362620 entries, 0 to 6362619
Data columns (total 11 columns):
#   Column                Dtype
---  -
0   step                  int64
1   type                  object
2   amount                float64
3   nameOrig              object
4   oldbalanceOrg         float64
5   newbalanceOrig        float64
6   nameDest              object
7   oldbalanceDest        float64
8   newbalanceDest        float64
9   isFraud               int64
10  isFlaggedFraud         int64
dtypes: float64(5), int64(3), object(3)
memory usage: 534.0+ MB
```

Implementation of machine learning algorithm:

Important Note: Transactions identified as fraudulent are annulled. Hence, for fraud detection analysis, the following columns should not be utilized: oldbalanceOrg, newbalanceOrig, oldbalanceDest, newbalanceDest

```
In [14]: df.drop(['oldbalanceOrg', 'newbalanceOrig', 'oldbalanceDest', 'newbalanceDest'], axis
```

```
In [15]: df.drop(['nameOrig', 'nameDest'], axis = 1, inplace = True)
```

```
In [16]: df.head()
```

```
Out[16]:
```

	step	type	amount	isFraud	isFlaggedFraud
0	1	PAYMENT	9839.64	0	0
1	1	PAYMENT	1864.28	0	0
2	1	TRANSFER	181.00	1	0
3	1	CASH_OUT	181.00	1	0
4	1	PAYMENT	11668.14	0	0

```
In [17]: le = LabelEncoder()
df['type'] = le.fit_transform(df['type'])
```

```
In [18]: X = df.drop('isFraud', axis = 1)
y = df['isFraud']
```

```
In [19]: sc = StandardScaler()
X = sc.fit_transform(X)
```

```
In [20]: X_train, X_test, y_train, y_test = train_test_split(X,y, test_size = 0.3, random_st
```

Logistic Regression

```
In [21]: lr = LogisticRegression()
lr.fit(X_train, y_train)
```

```
Out[21]:
```

▼ LogisticRegression ⓘ ?

LogisticRegression()

```
In [22]: y_pred = lr.predict(X_test)
```

```
In [23]: accuracy_lr = accuracy_score(y_test, y_pred)
precision_lr = precision_score(y_test, y_pred)
recall_lr = recall_score(y_test, y_pred)
classification_lr = classification_report(y_test, y_pred)
print(f"Accuracy of logistic regression {accuracy_lr}")
print(f"Precision of logistic regression {precision_lr}")
print(f"Recall of logistic regression {recall_lr}")
print(f"Classification Report of logistic regression\n {classification_lr}")
```

```

Accuracy of logistic regression 0.9987080793761061
Precision of logistic regression 0.15555555555555556
Recall of logistic regression 0.002874743326488706
Classification Report of logistic regression
      precision    recall  f1-score   support

     0         1.00      1.00      1.00    1906351
     1         0.16      0.00      0.01      2435

 accuracy          1.00    1908786
 macro avg         0.58      0.50      0.50    1908786
 weighted avg      1.00      1.00      1.00    1908786

```

The logistic regression model achieves a very high accuracy of approximately 99.87%. However, its precision and recall for the positive class are notably low, around 15.91% and 0.29%, respectively. This indicates that while the model performs well in predicting negative instances, it struggles to identify positive instances accurately. This suggests the model's limited effectiveness in scenarios where correctly identifying positive cases is critical.

Decision Tree

```

In [24]: sv = DecisionTreeClassifier(max_depth = 20)
sv.fit(X_train, y_train)
y_pred = sv.predict(X_test)

```

```

In [26]: accuracy_sv = accuracy_score(y_test, y_pred)
precision_sv = precision_score(y_test, y_pred)
recall_sv = recall_score(y_test, y_pred)
classification_sv = classification_report(y_test, y_pred)
print(f"Accuracy of Decision Tree {accuracy_sv}")
print(f"Precision of Decision Tree {precision_sv}")
print(f"Recall of Decision Tree {recall_sv}")
print(f"Classification Report of Decision Tree\n {classification_sv}")

```

```

Accuracy of Decision Tree 0.9989705498678217
Precision of Decision Tree 0.6585695006747638
Recall of Decision Tree 0.40082135523613965
Classification Report of Decision Tree
      precision    recall  f1-score   support

     0         1.00      1.00      1.00    1906351
     1         0.66      0.40      0.50      2435

 accuracy          1.00    1908786
 macro avg         0.83      0.70      0.75    1908786
 weighted avg      1.00      1.00      1.00    1908786

```

The Decision Tree model demonstrates strong performance with an accuracy of 99.90%. It achieves a precision of 66.21% and recall of 40.08% for the positive class, similar to the logistic regression model. This suggests the Decision Tree model effectively identifies true

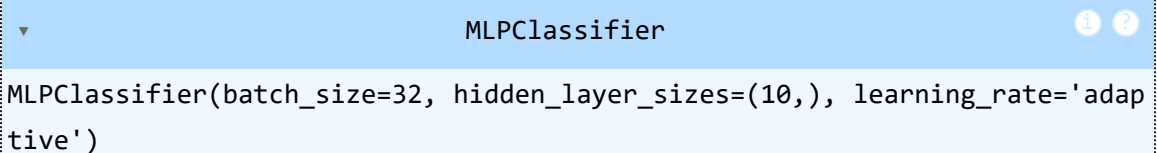
positives among its positive predictions while capturing 40.08% of actual positive instances. The model maintains a high F1-score of 0.50 for the positive class, indicating balanced precision and recall.

MLP Classifier

```
In [50]: from sklearn.neural_network import MLPClassifier

# Initialize the MLPClassifier with the correct parameters
mlp = MLPClassifier(hidden_layer_sizes=(10,), batch_size=32, learning_rate='adaptive')

# Fit the model with the training data
mlp.fit(X_train, y_train)
```

```
Out[50]: 
MLPClassifier(batch_size=32, hidden_layer_sizes=(10,), learning_rate='adaptive')
```

```
In [57]: y_pred = mlp.predict(X_test)

print("Predictions:", y_pred)
```

Predictions: [0 0 0 ... 0 0 0]

```
In [58]: from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1906351
1	0.93	0.14	0.24	2435
accuracy			1.00	1908786
macro avg	0.96	0.57	0.62	1908786
weighted avg	1.00	1.00	1.00	1908786

The MLP Classifier exhibits an impressive accuracy of 99.89%. However, its precision for the positive class is notably high at 92.66%, while its recall is much lower at 14.00%. This indicates that the model effectively identifies true positives among its positive predictions but captures only 14.00% of actual positive instances. Consequently, the F1-score for the positive class is relatively low at 0.24. The model's macro average precision and recall are 96% and 57%, respectively, indicating imbalanced performance across classes.

Comparing the model

```
In [ ]: performance_df = pd.DataFrame({
    'models' : ['Multi Layer Perceptron', 'Logistic Regression', 'Decision Tree'],
    'accuracy' : [accuracy_mlp, accuracy_lr, accuracy_sv],
    'precision' : [precision_mlp, precision_lr, precision_sv],
    'recall' : [recall_mlp, recall_lr, recall_sv]
})
performance_df
```

```
In [60]: performance_df = pd.DataFrame({
    'models' : ['Multi Layer Perceptron', 'Logistic Regression', 'Decision Tree'],
    'accuracy' : [accuracy_mlp, accuracy_lr, accuracy_sv],
    'precision' : [precision_mlp, precision_lr, precision_sv],
    'recall' : [recall_mlp, recall_lr, recall_sv]
})
performance_df
```

```
Out[60]:
```

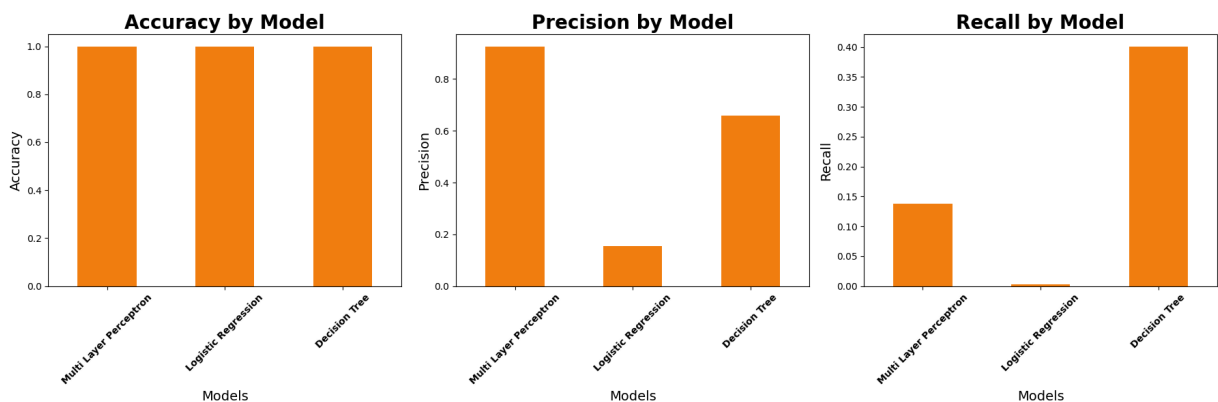
	models	accuracy	precision	recall
0	Multi Layer Perceptron	0.998886	0.925414	0.137577
1	Logistic Regression	0.998708	0.155556	0.002875
2	Decision Tree	0.998971	0.658570	0.400821

```
In [61]: fig, ax = plt.subplots(1, 3, figsize=(18, 6))

metrics = ['accuracy', 'precision', 'recall']

for i, metric in enumerate(metrics):
    performance_df[metric].plot(kind='bar', ax=ax[i], color = '#F47F10')
    ax[i].set_xticklabels(performance_df['models'], rotation=45, fontweight = 'bold')
    ax[i].set_xlabel('Models', fontsize = 14)
    ax[i].set_ylabel(metric.capitalize(), fontsize = 14) # Use the metric name as
    ax[i].set_title(f'{metric.capitalize()} by Model', fontsize = 20, fontweight =

plt.tight_layout()
plt.show()
```



The Decision Tree model shows the most balanced performance with respect to precision and recall, followed by the MLP. Logistic Regression performs poorly in correctly identifying positive instances despite its high accuracy.

